Improved Bounds for Strongly-Connected Components in the Streaming Model

[redacted] [redacted]

January 15, 2019

Abstract

In this paper we further examine theoretical properties of the Look-Select Algorithm [5] which is an online algorithm to determine the number of Strongly Connected Components in a graph. Previously Laura et. al, proved the worst case properties of this algorithm and showed that the experimental performance of the Look-Select Algorithm exceeded their worst case results as well as proven upper bounds for runtime. We present a new random directed graph model that lets us specify the number of components and sizes of the components, and density of our random graph which gives us a good way to examine the expected number of passes for the Look Select Algorithm. We show under two possible interpretations of a "Random Directed Graph" the expected number of passes of our algorithm is upper bounded by $O(log(n))$ as compared to n which was presented in Laura et. al. We also provide a reference implementation[1] of the Look-Select algorithm, and compare our algorithm with several efficient offline implementations for various random graphs. Finally, we investigate the height of the internal tree throughout execution, and propose several conjectures and open problems based on this experimental evidence.

1 Introduction

Computing strongly-connected components is one of the fundamental primitives in graph algorithms. In the sequential setting, Tarjan's linear-time algorithm based on depth-first search is considered to be the canonical method for computing strongly-connected components [10].

However, present-day applications for various graph algorithms often must operate on graphs with possibly millions of nodes (i.e. compute the connectivity of a subset of the Facebook graph). In this setting, sequential algorithms are either prohibitively slow, or the entire graph cannot fit in-memory. In this case, we turn to the *streaming* setting, where we know the nodes a-priori and receive the edges $e_1, e_2, \ldots, e_m$ sequentially. Sometimes, we slightly modify the model, for instance allowing the processor to write a subset $\{e_{i_1}, \ldots, e_{i_k}\} \subseteq E$ of the edge stream to an auxillary stream to be processed in a subsequent phase.

While the field of graph algorithms is very mature (with linear-time/efficient algorithms known for a large set of problems), the streaming setting is far less well-studied. The current state-of-the-art (deterministic) algorithm for computing strongly-connected components in the streaming setting is due to Laura & Santaroni [5]. The authors describe a *look-and-select* approach which classifies

[redacted]

This paper represents our own work in accordance with university regulations [redacted]

[1] [redacted]

edges into one of several categories ("look" phase) and selects a subset of edges to re-insert into the edge stream for a subsequent pass ("select" phase).

However, the authors describe a relatively weak worst-case amortized time bound which is dependent on the maximum height of a modified DFS tree that the algorithm maintains across all streaming passes (we will denote this as $h_{\max}$). In addition, the authors' experimentally tested their algorithm based on graphs generated by a particular pseudo-random procedure.[2] These experimental results suggest that the worst-case bound is quite pessimistic, in that they suggest the actual height of the internal tree is much closer to $\log n$ than n, as implied by the worst-case bound.

1.1 Our contribution

In this paper, we confirm the authors' hypothesis. In particular, we show that in the average-case, the Look-Select algorithm terminates in at most $O(\log n)$ streaming passes. This is a large improvement over the worst-case bound given by [5], which bounds the number of streaming passes above by $O(h_{\max})$.

1.2 Related work

In the sequential setting, Tarjan's depth-first search-based approach [10] is considered the canonical method for computing strongly-connected components. Several other popular linear-time methods also exist [3, 8].

In the streaming setting, there are a number of well-studied and closely related models. Henzinger, et. al. [4] introduced the general *data-stream* model, in which elements $x_1, \ldots, x_n$ arrive in a sequence, with the restriction that the algorithm has $O(\log n)$ working memory. Feigenbaum, et. al. [2] introduced the *semi-streaming* model, where $O(n \operatorname{poly}(\log n))$ working memory is permitted. Demetrescu, et. al. [1] introduced the model, which permits writing to an auxillary edge stream to be processed in a subsequent phase.

While there are a number of results for undirected graph problems, including efficient streaming algorithms for exact and approximate connected components, minimum spanning tree, and others (for a detailed summary, see the survey [7]). In the directed context, however, very few results are known. Apart from several algorithms for directed path problems due to Henzinger, et. al. [4], the only other known algorithm in the *directed* streaming model is the Look-Select algorithm of Laura & Santaroni [5].

In the context of our work, it also is worth mentioning the related notion of *(fully)-dynamic* data structures. These data structures maintain either a relation R or some property P on a graph G over a sequence of edge insertions and removals. Although the running-times of these algorithms may be far more efficient than algorithms in the streaming setting, few obey the $O(n \log n)$ space constraint imposed by the data stream model.

1.3 Outline

The remainder of the paper is organized as follows; in Section 2, we will introduce notation, some basic definitions, and our streaming model. In Section 3, we will describe the Look-Select algorithm and re-state some basic results from [5]. In doing so, we will also address several issues

[2] Although they allude to this procedure for generating their test graphs, the authors never actually describe the procedure. This is a major motivation for studying the ramdom graph model described in the next section.

in the authors' original paper. In Section 4, we introduce our random graph model. In Section 5, we use this model to derive a new average-case bound for the running time of Look-Select. In Section 6, we present experimental results on several benchmarks. Finally, in Section **??** we will summarize our results and offer some directions for future work.

2 Definitions & preliminaries

Throughout this paper, $G = (V, E)$ denotes a directed graph with $|V(G)|$ vertices and $|E(G)|$ edges. We will denote an edge by $(u, v) \in E$. We will write $u \rightsquigarrow v$ to denote the existence of a directed path from u to v in G. We write $u \leftrightsquigarrow v$ if both paths exist.

We let $\sigma : E(G) \to [m]$ denote an *ordering* on the edge set $E(G)$. That is, $\sigma(E(G)) = \{e_{\sigma_1}, e_{\sigma_2}, \ldots, e_{\sigma_m}\}$ such that $\sigma_i > \sigma_j$. If $\sigma(E(G))$ is clear from context, we will denote this simply by σ. Finally, we let $T : \mathcal{A}, \mathcal{I} \to \mathbb{N}$ denote the *running-time* of an algorithm $A \in \mathcal{A}$ on an input $I \in \mathcal{I}$.

Finally, for any rooted tree T, for any $u \in V(T)$, the *height* $h_T(u)$ is given by the height of the subtree of T rooted at u. We denote the parent node of any $u \in V(T)$ as $F(u)$.

2.1 Graph-theoretic preliminaries

Definition 2.1. Given a directed graph $G = (V, E)$, $C \subseteq V$ is a *strongly-connected component* of G if $\forall u, v \in C$, $u \leftrightsquigarrow v$. Let $\mathrm{SCC}_G = \{C \subseteq V | \forall u, v \in S, u \leftrightsquigarrow v\}$ denote the set of SCCs in G. If $C \in \mathrm{SCC}_G$ also satisfies the property that there is no $C' \in \mathrm{SCC}_G$ such that $C \subset C'$, then we call C a *maximal strongly-connected component*. Let $\mathrm{SCC}_{\max}$ denote the set of maximal SCCs.

It is clear that for any $M \in \mathrm{SCC}_{\max}$, M is the union of some $C_1, \ldots, C_k \in \mathrm{SCC}_G$ strongly-connected components. The Look-Select algorithm described in the next section iteratively maintains some subset $\mathcal{C} \subseteq \mathrm{SCC}_G$ during each phase, and contracts them until we are left with a new set $\mathcal{C}' \subseteq \mathrm{SCC}_{\max}$.

2.2 Streaming models

Finally, we introduce some basic definitions about our streaming model.

Definition 2.2. For $G = (V, E)$, an *edge stream* S is simply any ordering $\sigma(E(G))$.

Definition 2.3. The *W-stream* model of computation provides our algorithm with an initial stream S_0, and external memory $\mathcal{M}$ and allows it to operate in a series of passes. In pass i our algorithm can read from a stream S_{i-1} and write to a FIFO queue S_i (distinct from $\mathcal{M}$) that will serve as the input stream for the next pass. The algorithm completes when the stream for its pass is empty. The model requires that $|\mathcal{M}| = O(n \log n)$.

Although we are limited to $O(n \log n)$ space and do not have as much flexibility for what we can encode in $\mathcal{M}$ as in the semi-streaming model, the ability to write to an auxillary stream provides an opportunity to design a memory-efficient algorithm for computing strongly-connected components in G. Finally, we make a few statements regarding the flexibility of the W-stream model.

Remark 2.1. The W-stream model can simulate many other streaming models (the class of W-stream algorithms that complete after one pass all are valid algorithms under the classical data-stream model).

Remark 2.2. This type of model can effectively simulate a system in which low latency memory is small, and high-latency memory is very large. For instance, systems that typically run distributed graph-processing engines like GraphX or Pregel follow these hardware characteristics.

3 The Look-Select algorithm

In this section, we describe the LOOK-SELECT algorithm presented in [5]. We will first present the overall idea of the algorithm's "look" and "select" phases, and then provide a more detailed implementation. In doing so, we also correct an error in the algorithm's original presentation.

3.1 Preliminaries

The input to the LOOK-SELECT algorithm is given by the triple (n, E, σ), where n is the number of vertices, E is the edge set, and $\sigma(E(G))$ is an ordering for E. We let $S_0 = \sigma(E(G))$ denote the initial edge stream.

The algorithm maintains two data structures throughout its execution; a rooted tree $\mathcal{T}$ and a Union-Find structure $\mathcal{U}$.[3] They obey the following properties;

Condition 1 ($\mathcal{T}$): $\mathcal{T}$ is a rooted, directed tree, in the sense that there exists some unique root $r \in V(\mathcal{T})$ such that $V(\mathcal{T}) = V(G) \bigcup \{r\}$, and for all other $u \in V(\mathcal{T})$, there is only one path $u \rightsquigarrow r$ (including the empty path $r \rightsquigarrow r$). This tree will store information about the SCCs. Each $v \in V(\mathcal{T})$ represents a set of vertices that is strongly-connected.

More formally, we say that there exists a mapping $\phi : V(\mathcal{T}) \to \mathcal{P}(V(G))$ where $\phi(t) = S \subseteq V$, and for all $u, v \in S$, $u \leftrightsquigarrow v$ (it is clear ϕ forms an equivalence class on $V(G)$). There also exists a mapping ϕ_e, that is defined as follows: for $(u, v) \in E(G)$, $\phi_e(u, v) = (\phi(u), \phi(v)) \in E(\mathcal{T})$ We see that ϕ allows us to translate vertices $v \in V$ into SCCs of G, while ϕ_e allows us to translate an edge $(u, v) \in E(G)$ to $(u', v') \in E(\mathcal{T})$.

The initial state of $\mathcal{T}^{(0)}$ is as follows; $V(\mathcal{T}^{(0)}) = V(G) \bigcup \{r\}$, and $E(\mathcal{T}^{(0)}) = \{u \in V(G) | (r, u)\}$ (the dummy root r with children $v_1, \ldots, v_n$ and an edge between each (r, v_i).

Condition 2 ($\mathcal{U}$): We represent the mappings ϕ_e and ϕ^{-1} via a Union-Find data structure $\mathcal{U}$. Our use of ϕ^{-1} is arguably an abuse of notation, but since ϕ is an equivalence relation on $V(G)$, we let $\phi^{-1}(v)$ denote the label of v under this equivalence class. $\phi^{-1}(v)$ can be thought of as a "canonical" label for the SCC containing v. The initial state of $\mathcal{U}^{(0)} = \{\{v_1\}, \{v_2\}, \ldots, \{v_n\}\}$ (with each vertex occupying its own strongly-connected component when no edges have been seen).

3.2 Edge cases

Next, we define a series of classes that the LOOK-SELECT algorithm classifies each e_i into. The "Look" phase of the algorithm examines each e_i and classifies it according to the following definitions. Then, the "Select" phase selects a subset of the edges to insert into the next stream, updating $\mathcal{T}$ and $\mathcal{U}$ accordingly.

All of these edge classes are labels that we give to $h \in E(G)$ with respect to the state of $\mathcal{T}$. From here on we will denote $\mathcal{T}^{(i)}$ as the state of our tree at some index i (it will become more clear later what precisely the i denotes. For a directed edge $h = u \to v \equiv (u, v) \in V(G) \times V(G)$.

[3] This is also referred to as a disjoint-union structure.

(1) Backward: $\phi_e(h)$ if added to $\mathcal{T}^{(i)}$ would form a cycle (that is, v is an anscestor of u).

(2) Cross-Forward: $h(u) \geq h(v)$, and u and v are in different sub-trees.

(3) Cross-Non-Forward: $h(u) < h(v)$, and u and v are in different sub-trees.

(4) Forward: v is a descendent of v in $\mathcal{T}^{(i)}$ (there is already a directed path

(5) Self-Loop: $\phi^{-1}(u) = \phi^{-1}(u)$

We omit a simple proof that these classes of edges partition $E(G)$.

3.3 The Algorithm

Now, we can simply state the LOOK-SELECT algorithm. We start with stream S_0 and $\mathcal{T}^{(0)}$, $^{(0)}$ as described above. Given a stream S_k, for each edge $e = (i, j)$, and we take its class (from above) which we will notate as some value from 1-5. Let $(u, v) \in V(\mathcal{T})$ denote $(u, v) = \phi_e(i, j)$ (i.e. e translated into a tree-edge).

if **Case 0**: If $F(v) = r$, then remove (r, v) from $\mathcal{T}^{(i+1)}$ and add (u, v)
Case 0a: If (u, v) now satisfies Case 1, then go to Case 1.
else if **Case 1**: Form $\mathcal{T}^{(i+1)}$ by contracting vertices along $u \rightsquigarrow v$ and update $\mathcal{U}$ accordingly. If $F(u) = \perp$, then set $F(u) = r$ and add (r,u) to $\mathcal{T}^{(i+1)}$
else if **Case 2**: Place (u, v) in S_{k+1}
else if **Case 3**: Remove $(F(v), v)$ from $\mathcal{T}^{(i+1)}$ and add (u, v). Add $(F(v), v)$ to S_{k+1}.
else if **Case 4 or 5**: Drop edge

We repeat this procedure until either (a) $|S_{k+1}| = 0$ (i.e. there are 0 elements in the next stream), or (b) $\mathcal{T}^{(i)} = \mathcal{T}^{(i+1)}$ (i.e. $\mathcal{T}$ did not change between phase i and $i + 1$).

We briefly note that we explicitly added **Case 0a** to our pseudocode for LOOK-SELECT since the algorithm is incorrect if it is omitted. We illustrate this with a simple counter-example below:

Observation 3.1. *If $G = C_3$ (i.e. the directed 3-cycle) with vertices $\{a, b, c\}$, then* LOOK-SELECT *as presented in [5] incorrectly terminates. In particular, an illegal back-edge is introduced into $\mathcal{T}^{(i)}$ that is never subsequently contracted.*

Proof. Consider the edge sequence $(a, b), (b, c), (c, a)$. $\mathcal{T}^{(0)}$ contains edges from r to each of the $v \in \{a, b, c\}$. If we process the edges in this sequence while omitting **Case 0a**, $\mathcal{T}^{(i)}$ for $i > 0$ will violate Condition 1, namely that $\mathcal{T}^{(i)}$ contains a cycle. The key observation is that **Case 0** runs before any of the other cases, and the algorithm ignores them if $F(v) \neq r$. Thus, if the output of Case 0 is a backward edge (Case 1), then it remains uncontracted and the tree's state is corrupted. However, if Case 0a runs, then the invariant is preserved. □

The figure above illustrates the counter example on C_3. The top row is the algorithm with Case 0a omitted, while the bottom shows the correct execution sequence. Note that we also explicitly specify to add an edge from (r, u) if u is left without a parent following contraction. Thus, the state of the tree is correctly preserved and we can proceed with our analysis.

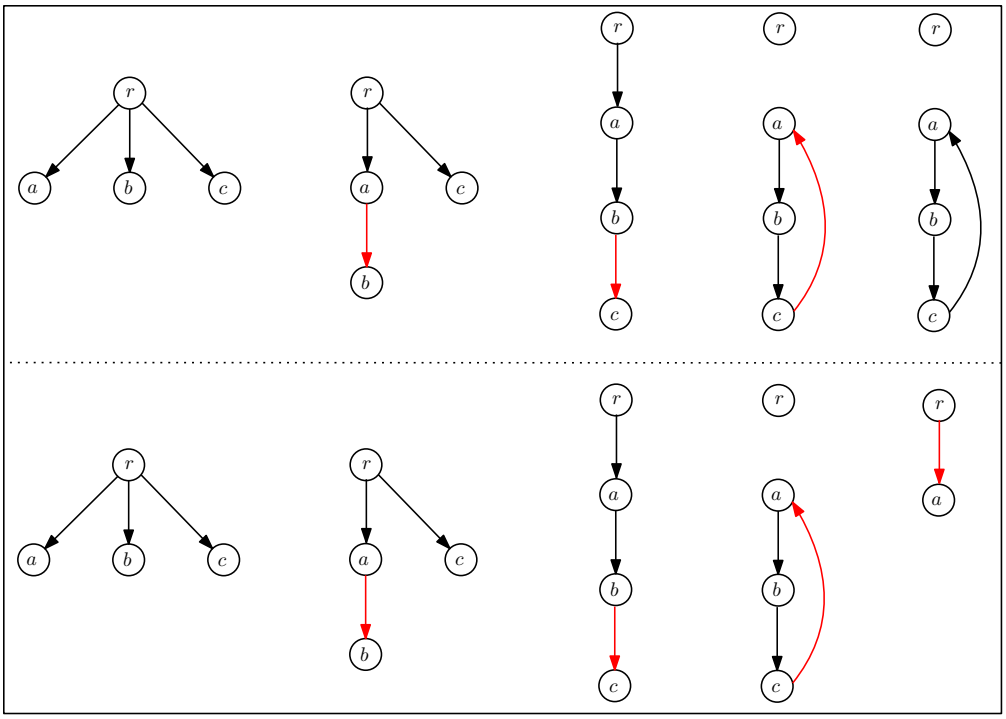


Figure 1: (Top) Incorrect execution of LS on C_3 (Bottom) Correct execution on C_3. Red edges are new in the given phase.

4 A random graph model

In order to examine the performance of the Look Select Algorithm under expectation, we need to define a random graph model which will be the distribution of graphs which we will consider the algorithm's performance on. This statement deserves a little more definition:

Remark: The Look-Select algorithm takes as input an stream of edges $(e_0, ..., e_m), e_i \in E(G)$, where $|V(G)| = n$. We can treat this input as having two parameters - the graph G and the edge ordering $\sigma : E(G) \to [m]$.

Our goal is to understand the performance of LS under various random input conditions. The first condition we are concerned with is the case of fixed n, randomized E, and randomized σ. The terms 'randomized' require more definition.

Def: We define the random variable Σ as the function that takes on a random permutation of E. This distribution is defined with $\mathbb{P}[(e_1, ..., e_m)] = \frac{1}{m!}$.

Remark: Our distribution over E is equivalent to the problem of generating a random graph. Defining our model from which our random graph is drawn is important to our analysis as we want to choose a distribution that doesn't trivialize our problem and preserves properties of our graph that our algorithm is looking for. For example if we have an algorithm to determine whether a graph has property b, but graphs with property b are improbable, then it is likely the case that the

trivial algorithm that returns false always will perform well under expectation.
(1) We want to generate a distribution of graphs parameterized by n, $[p_0, ..., p_k]$, where $p_i > 0, ||\hat{p}||_1 = 1$, and d, which is our average degree. Graphs from this distribution will have k strongly connected components, where component i has $p_i n$ vertices. We choose these parameters because we want to analyze the performance of our algorithms under expectation on the property we are testing for (as suggested by Spielman [9]).
(2) There aren't any established ways to construct such a distribution of graphs explicitly (ignoring rejection sampling or other methods that require a high probability of a graph with our property appearing from some other distribution we can construct). With the following results we will show a reasonable distribution of graphs with our properties we will prove results about the LS algorithm on.

Algorithm I Given m vertices $\{v_1, ..., v_m\}$ we generate a strongly connected component in the following way.
(i) add the directed m cycle over all the vertices. We will choose the order given in the set $v_1 \to v_2 \to ... \to v_m \to v_1$. The order of this cycle doesn't matter (all such cycles are the same under isomorphism)
(ii) for every pair $(i, j), i \neq j$, we draw the directed edge from $v_i \to v_j$ with probability q

Claim: Any G which is an output of Algorithm I, is strongly connected and has $m + (m)(m-1)q$ edges on expectation.
Proof: This is rather trivial to show, as the first m edges come from the cycle, and for every pair of edges we have a probability q of drawing the edge thus giving us our result via linearity of expectation. We observe that because all vertices are on our cycle we know our graph is strongly connected.

We claim that this is an important procedure for us for a few reasons, but first we will address the fact that there are many strongly connected components that will never be generated by this procedure, namely the class or SCCs with max cycle length $< m$. In fact this is exactly the family of excluded graphs. We can prove this:

Proof: Any graph $G, |V(G)| = m$, with max cycle $< m$ can clearly not be created as that would contradict step (i) of our algorithm. This is the easy direction. Next we will show that if the max cycle size is indeed m with a graph on m vertices, we can create any such graph.
Let us take $G, |V(G)| = m$, with max cycle $C, |C| = m$. It follows that $V(G) = V(C)$. Consider $G' = G - E(C)$, and we observe that we can create G by adding the edges of G' to C, which we can do via (ii). Thus if G has $n > m$ edges, our procedure generates G with probability $(n-m)^q (m^2 - n - m)^{(1-q)}$ which is nonzero finishing our proof.

Remark: We will now discuss why this distribution is still reasonable for us to examine the LS algorithm on. From the background we recall that the difficulty of an instance of our problem is related to the size of the maximum Deep-Cycle in our graph. Thus the fact that our distributions of graphs seem to have larger cycles than the uniform distribution doesn't trivialize our problem. More formally, the random variable C_1 which takes on the max cycle size of a "uniform" distribution over SCCs on m vertices. If we take C_2 as the max cycle size of our distribution (which will be m), we can see it clearly stochastically dominates C_1.

Next we need a way to use Alg I as a subroutine to generate a graph with multiple SCCs.

Algorithm II This algorithm will produce a random DAG (Directed Acyclic Graph) on k vertices. We proceed via this simple algorithm. We take our vertex set $\{v_1, ..., v_k\}$.
(1) We choose a random ordering U of the set $\{(i,j) \in [k] \times [k] | i \neq j\}$
(2) We iterate through elements of U and for each subsequent (i', j'), if there isn't a path between $v_{i'}$ and $v_{j'}$ already in the graph then we add an edge from $v_{i'} \to v_{j'}$ with probability x.

Claim: This distribution assigns nonzero probabilities only to DAGs, and any DAG on k vertices is achievable (has nonzero probability) under this method.
Proof: Our invariant (2) shows that the addition of each edge will never create a directed cycle. Thus only DAGs have nonzero probability. Now take some arbitrary DAG D, and take its edges $E(D), |E(D)| = c$. The probability of this graph being generated by our algorithm is lower bounded by the probability that regardless of U we choose to put an edge on the vertices of D and no edge on any of the others. This will satisfy our invariant in (2) as D has no cycles. This is

$$x^c(1-x)^{k(k-1)-c} > 0$$

Algorithm III
(1) Given $\hat{p}$ as defined earlier we separate n vertices into k groups, of sizes $p_i n$. For each of these are generate a SCC_i on $p_i n$ vertices as per Algorithm I.
(2) We create a random DAG on k vertices $\{u_1, ..., u_k\}$ to form a graph H as per Algorithm II.
(3) For every $(i,j), i \neq j$ such that $u_i \to u_j$ exists in H, we perform the following for SCC_i, and SCC_j. For every $v \in SCC_i, w \in SCC_j$, with probability f, we add an edge from $v \to w$.
(4) We output the SCC_is along with the edges between them as our graph G.

This is the model we will generate our graphs with. This is a useful formulation of our distribution, as we have 3 'levers' controlling the sparsity/density of the graph (a property of many good random graph models). q controls the intra-component density. f and x control the density of edges between components. Additionally, we don't trivialize our problem with this distribution because we both maintain the number of SCCs and the size of the SCCS (a property we are testing), as well as preserve what makes the problem hard (deep cycles – more on this soon)

5 Average-case analysis

We now consider a random graph G generated by Algorithm III, and we will examine the performance of our algorithm on this graph. We fix $\hat{p}$, and q, x, f (which implicitly are levers that control the average degree).
Claim: Every Deep Cycle in $\mathcal{T}$ has its corresponding vertices in the same connected component in G.
Proof: We know from our definition of a Deep Cycle that its edges in T form a cycle. Each vertex of T corresponds to a set of vertices that are strongly connected in G. Let us call $t_1, ..., t_m$ the vertices of T in our deep cycle, and let us call $v_1, ..., v_m$ as the vertex sets that are strongly connected represented by these $t_1, ..., t_m$. WLOG, we say that the edges in the deep cycle with respect to T connect $t_1 \to t_2 \to ... \to t_m \to t_1$. If we consider the edges in the deep cycle as edges in G we

have $t_{1a} \to t_{2b}, t_{2a} \to t_{3b}....t_{ma} \to t_{1b}$, where $t_{ia}, t_{ib} \in v_i \subset V(G)$. It follows that since $t_{ia}, t_{ib} \in v_i$, they are strongly connected to each other so we know there exists a path within the components $t_{1a} \to t_{2b} \rightsquigarrow t_{2a} \to t_{3b}....t_{ma} \to t_{1b} \rightsquigarrow t_{1a}$, forming a directed cycle in G. We observe that this implies that since we have a cycle between some elements of each v_i in G, all the v_i are strongly connected to each other (we can form the paths by piecing together part of the metacycle we found with the path to the corresponding t_{ia} or t_{ib} within v_i which exists. An alternate proof can come from the nature of SCCs as equivalence relations.

We can use this to trivially tighten our bound on the size of the max deep cycle to form our first improvement on Laura et al.'s results:
Corrollary: For any Deep-Cycle $D, |D| = k$, it holds that $k \leq \min\{max(T), m\}$, where $hmax(T)$ is the maximum height reached by the tree T, and m is the size of the maximum connected component.
Although this was not a difficult bound to derive, it is important because it shows the necessity of the parameters we chose for our family of random graphs. We will show its importance by showing how a naive distribution makes this problem easy
Claim: If we take G to be randomly selected from the family of Erdos-Reyni-type digraph, where for every (i, j) we flip a coin with probability $p < \frac{1}{2n}$ as to whether or not to add an edge, with high probability we have

$$|D| < O(log(n))$$

Proof: This is an application of Erdos-Reyni's results from their original paper where they prove that for an undirected E-R graph $G(n, \frac{1}{n})$, G will almost surely have no connected component larger than $O(log(n))$. We observe that for di-graphs, we can imagine the E-R generation for digraphs to be choosing an undirected graph, and then flipping a coin to determine the direction of the edge. Thus the max SCC in our digraph D can be no larger than the max component size in G. Thus we know we will w.h.p have no $SCC, |SCC| > O(log(n))$
Corollary: On such an ER Digraph with high probability LS will take $O(log(n))$ passes

We have not only shown an interesting bound for E-R-type graphs, but we have also shown the importance of choosing a property to preserve in our distribution. For graphs generated by Algorithm III, we assign the number and size of the SCCs.

Def We define graphs generated by Algorithm III, as component-specified random graphs (CSRG). For the rest of this analysis we will assume our graph G is drawn from a CSRG distribution

We examine a deep cycle of size $r < m$ in some SCC $U \subset G, |U| = m$. Specifically, we want to know what the probability of r appearing is.
Calculating this exact probability is difficult, but we can get an upper bound on the probability.
Def: We define a Deep Cycle Candidate $\mathcal{DC}$ as a subgraph of G that has the potential to be a deep cycle under the right σ. We have the nodes of a $\mathcal{DC}$ which are sets of vertices in G which will be nodes in our tree $\mathcal{T}$
(1) We take a component with m vertices, and consider a $\mathcal{DC}$, which has $n \leq m$ nodes.
(2) For a $\mathcal{DC}$ with n nodes, we observe from our construction that there is a

$$\frac{m^2}{n^2}\frac{1}{m^2} = \frac{1}{n^2}$$

N	min(Kos, Gab)	LS
15	0.05	0.11
20	0.05	0.23
25	0.1	0.24
100	0.12	0.3
1000	0.45	0.81
5000	0.67	1.1
10000	1.02	2.9
100000	31	65
500000	43	88
1000000	OOM	112
2000000	OOM	201
5000000	OOM	388

Table 1: Timing results (in s). OOM denotes out-of-memory.

probability that a valid back-edge is placed (in the subsequence of edges that are between the nodes of our cycle). If this isn't the case then the Deep Cycle wouldn't exist. Thus the probability of a deep cycle must be upper bounded by $\frac{1}{n^2}$.
This gives us an upper bound on an expected max deep cycle as:

$$\sum_{n=2}^{m} \frac{1}{n^2} n \prod_{j=1}^{n} (1 - \frac{1}{(j-1)^2}) \leq \sum_{i=1}^{m} \frac{1}{i} \leq c \log(m)$$

Where the final inequality comes from the harmonic sum bound.
Corollary: In the CSRG model, LS takes $\leq O(log(n))$ passes to terminate under expectation

6 Experimental results

We also implemented the Look-Select algorithm in order to empirically measure the performance of this algorithm. We ran benchmarks for several randomly generated graphs with a fixed number of strongly-connected components (according to the pseudo-random procedure outlined in Section 4).

We note that though the authors of [5] cite that they were able to implement a much faster version of this algorithm than we were, we also were much more resource-constrained (having to run them on our laptops). Thus, for our experimental setup, we ran all of our experiments on a 2.9 GhZ Intel Core i7 laptop with 16GB of RAM. In order to simulate a machine with low working memory, we ran our experiments on a JVM with the heap size limited to 2MB-100MB. We implemented the stream as an on-disk file, and used a `java.io.Buffered{Reader,Writer}` to write/consume the streams. This simulates a machine with a very small amount of RAM with a comparatively much larger disk space/stream space.

6.1 Timing results

We provide the timing results above. We compare our algorithm against the fastest offline strongly-connected components algorithm between the Kosaraju-Sharir and Gabow algorithms [8, 3] as im-

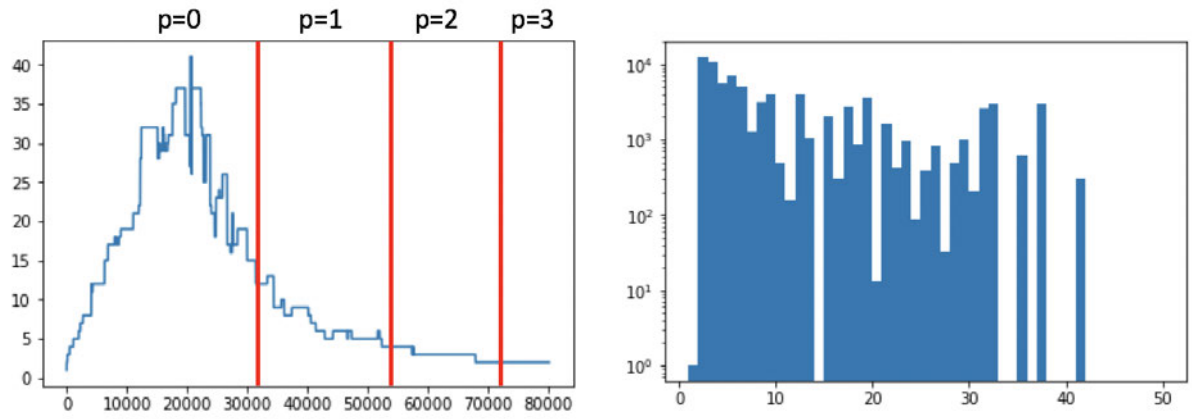


Figure 2: (a) Height of $\mathcal{T}$ across all iterations (b) Distribution of heights

plemented in `JGraphT`.[4] We note that LS performs comparably for small graphs, but ehibits a far greater amount of overhead than either the Kosaraju-Sharir or Gabow algorithms. This makes sense, given that not only do we maintain two internal data structures, but we also have all of the I/O overhead for reading/writing the streams to disk. However, for all graphs with 1 million or more nodes ran out of memory for `JGraphT` and only our LS algorithm ran.

6.2 Execution profiling

We also profile the execution of the algorithm and compute $h(\mathcal{T})$ at each iteration. We plot this as a function of the number of steps:

Here, we plot the height of the internal tree across each iteration, with p denoting the current streaming phase. We note here that the tree is largest when $p = 0$, and seems to experience an exponential decay in height after reaching its maximum approximately halfway through the first phase.

7 Future work

While we are able to bound p by $O(\log n)$, we are also particularly interested in alternative models for generating random graphs with specified information about the strongly-connected components (which may also be of independent interest). An avenue that we would like to explore is an approach that relies on incrementally "unrolling" the tree $\mathcal{T}$, starting from a dummy root r. The operations would roughly be the inverse of the tree operations (i.e. expand a cycle instead of contract, etc.). We conjecture that we can generate any arbitrary graph using this method.

If we are able to accomplish this, then we will in fact be able to use other tools to bound the size of the maximal deep cycle (i.e. bounding the number of such possible sequences in the edge stream). This may also give us the ability to analyze Look-Select and provide an *approximation* algorithm. Examining the graph, for $p = 0$, we can posit that the majority of stronglty-connected components are contracted during the first phase. Therefore, we make the following conjecture:

[4] `https://jgrapht.org/`

Conjecture 1. *If we halt the* Look-Select *algorithm after a single pass, then the SCCs contained in $\mathcal{U}$ gives an additive-error approximation for the number of actual SCCs. That is, let $\hat{c} = |\mathcal{U}|$. Let c denote the actual nubmer of SCCs. Then*

$$|\hat{c} - c| \leq \epsilon n.$$

References

[1] Camil Demetrescu, Irene Finocchi, and Andrea Ribichini. Trading off space for passes in graph streaming problems. *ACM Transactions on Algorithms*, 6(1):1–17, dec 2009.

[2] Joan Feigenbaum, Sampath Kannan, Andrew McGregor, Siddharth Suri, and Jian Zhang. On graph problems in a semi-streaming model. *Theor. Comput. Sci.*, 348(2-3):207–216, 2005.

[3] Harold N. Gabow. Path-based depth-first search for strong and biconnected components. *Information Processing Letters*, 74(3-4):107–114, may 2000.

[4] Monika Rauch Henzinger, Prabhakar Raghavan, and Sridhar Rajagopalan. Computing on data streams. In *External Memory Algorithms, Proceedings of a DIMACS Workshop, New Brunswick, New Jersey, USA, May 20-22, 1998*, pages 107–118, 1998.

[5] Luigi Laura and Federico Santaroni. Computing strongly connected components in the streaming model. In *Theory and Practice of Algorithms in (Computer) Systems*, pages 193–205. Springer Berlin Heidelberg, 2011.

[6] Andrew McGregor. Graph mining on streams. In *Encyclopedia of Database Systems*, pages 1271–1275. 2009.

[7] Andrew McGregor. Graph stream algorithms: a survey. *SIGMOD Record*, 43(1):9–20, 2014.

[8] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1):67–72, 1981.

[9] Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis. In *Lecture Notes in Computer Science*, pages 256–270. Springer Berlin Heidelberg, 2003.

[10] Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, jun 1972.